

ARCHIMESH

MANUAL

June 10, 2026

Contents

1	Introduction	6
1.1	What Archimesh offers	6
1.2	What this manual covers	6
1.3	Terminology.....	7
2	Getting Started	8
2.1	Prerequisites.....	8
2.2	Configuration.....	8
2.3	Starting infrastructure.....	8
2.4	Building and running the server	9
2.5	Creating a model.....	9
2.6	Building the client plugin	9
2.7	Building the admin UI	9
2.8	First collaborative session	10
2.9	Validation scripts.....	10
2.10	Stopping	10
2.11	Where to go next.....	11
3	Architecture	12
3.1	System purpose	12
3.2	High-level flow	12
3.3	Component layout.....	12
3.4	Server component map.....	13
3.5	Client plugin component map.....	13
3.6	Two kinds of state	13
3.7	Snapshot vs op-log	14
3.8	Folder model	14
3.9	Linear timeline	14

3.10	Deployment and operational considerations	15
4	Client Plugin	16
4.1	Plugin entry point	16
4.2	Session manager	16
4.3	EMF change capture	17
4.4	Operation mapping	17
4.5	Remote operation application	18
4.6	Offline outbox	18
4.7	Cache policy	19
4.8	Echo suppression	19
4.9	Menu commands	19
5	Server API.....	20
5.1	WebSocket endpoint	20
5.2	REST API	22
6	Persistence.....	25
6.1	Repository layer	25
6.2	Graph model	25
6.3	Neo4j constraints	28
6.4	Persistence invariants	28
6.5	Configuration	28
7	Messaging	30
7.1	Topic layout	30
7.2	Single-partition requirement	30
7.3	Producer	30
7.4	Consumer	30
7.5	Retention	31
7.6	Fallback broadcast	31
7.7	Configuration	31

8	CRDT and Merge Semantics.....	32
8.1	Causal identity.....	32
8.2	Merge types by domain	32
8.3	Delete semantics — tombstones.....	33
8.4	Notation field validation.....	33
8.5	Lock model	34
8.6	CRDT maturity	34
9	Authorization.....	35
9.1	Identity modes.....	35
9.2	Role model.....	35
9.3	Policy model	36
9.4	JWT and OIDC setup.....	36
9.5	Authorization diagnostics	37
9.6	Configuration summary	37
10	Administration and Supervision	38
10.1	Admin UI.....	38
10.2	Audit logging.....	38
10.3	Supervision checklist	39
11	System Management	40
11.1	Model lifecycle	40
11.2	Compaction	40
11.3	Export.....	41
11.4	Import.....	42
11.5	Rebuild	43
11.6	Tags.....	43
11.7	Integrity checks.....	44
11.8	Consistency checks.....	44
11.9	ACL management.....	44
11.10	Status monitoring	45

12	Troubleshooting	46
12.1	Diagnostic endpoints	46
12.2	Common problems	46
12.3	Log analysis.....	48
12.4	Recovery procedures	49
12.5	Performance tuning	49
13	Script Reference.....	51
13.1	Build and install	51
13.2	Development infrastructure.....	51
13.3	Validation and gate checks	51
13.4	Log monitoring	52
14	Wire Protocol and Schema Reference.....	53
14.1	Schema files.....	53
14.2	Message flow summary	53
14.3	Common type definitions.....	53
14.4	Op batch envelope.....	54
14.5	Notation JSON objects.....	54
14.6	Error codes.....	55
14.7	Export/import format	55

1 Introduction

Archimesh is a collaborative real-time modelling server for Archi, the ArchiMate modelling tool. It lets multiple modellers work on the same model simultaneously, with the server acting as the authoritative source of truth for the shared model timeline.

This manual is written for developers and operators who need to set up, run, supervise and troubleshoot an Archimesh deployment. It assumes familiarity with Java, Docker, Neo4j and Kafka at a working level.

1.1 What Archimesh offers

Archimesh provides five core capabilities:

- **Real-time collaborative editing.** Multiple Archi clients connect to one server, capture local edits as structured operations, and apply remote operations as they are committed. The server assigns a single authoritative revision order so every client converges on the same model state.
- **Authoritative model storage.** Neo4j stores both the current materialized model graph and the append-only operation log. The server is the source of truth; local Archi files are treated as cache projections.
- **CRDT-informed merge.** The server applies Last-Writer-Wins (LWW) per-field merges with causal clocks and tombstone protection for concurrent edits on the same diagram objects. Lease-based locks protect noisy notation edits (resize, move, bendpoints, style) from destructive interleaving.
- **Offline resilience.** When a client loses its connection, local edits are queued in a bounded persistent outbox and replayed in FIFO order after reconnect. Replayed ops are rebased to the latest server revision before send, and conflicting stale entries are deterministically dropped.
- **Operational visibility.** The admin API and admin UI expose model catalog, active sessions, recent activity, ACL management, audit trails, integrity checks, compaction control, and export/import for backup and migration.

1.2 What this manual covers

- Section 2 walks through building and running the server and client locally.
- Section 3 describes the system architecture, component layout and major data flows.
- Section 4 covers the Archi client plugin: EMF change capture, operation mapping, remote apply, offline outbox and cache policy.
- Section 5 documents the WebSocket and REST API endpoints, message types and request/response contracts.
- Section 6 explains the Neo4j graph model: materialized state, op-log, tags, tombstones and property clocks.
- Section 7 covers the Kafka topic layout, partitioning requirements and fan-out path.
- Section 8 details the CRDT merge semantics: LWW field merges, tombstone protection, OR-set collections and lock model.
- Section 9 documents identity modes, role normalization, per-model ACLs and the policy decision point.
- Section 10 explains the admin UI and operational supervision: overview, models, versions, sessions, access and audit.

- Section 11 covers system management: creating models, compactions, exports, imports, rebuilds, tags and integrity checks.
- Section 12 describes common failure modes, log analysis, diagnostic endpoints and recovery procedures.
- Appendix 13 is a reference for all scripts in the `scripts/` directory.
- Appendix 14 documents the wire protocol message types and JSON schema contracts.

1.3 Terminology

Model A named linear revision timeline. One `HEAD`, zero or more immutable tags, no branches. Identified by a stable `modelId`.

Operation (op) A single Archimesh change: create, update or delete of an element, relationship, view, view object, connection, folder or property.

Op batch A client-submitted group of operations sharing one `opBatchId` for idempotency. The server assigns one contiguous revision range to the batch.

Revision A monotonically increasing timeline position. Each op in a batch consumes one revision. The revision range is the authoritative ordering.

HEAD The current tip of the model timeline. Only `HEAD` is writable; tagged revisions are read-only.

Tag An immutable named pointer to a historical revision, stored with a snapshot of the model state at that point.

Materialized state The current model graph in Neo4j, kept in sync with the op-log. Used for fast joins, exports and admin inspection.

Outbox A bounded persistent FIFO queue on the client side that holds local edits while disconnected from the server.

Compaction Admin-controlled pruning of old op-log commits and merge metadata below a configurable retention watermark. Tombstones are retained for delete safety.

2 Getting Started

This section walks through building and running Archimesh locally. It assumes Docker Desktop (or Docker Engine with the Compose plugin) and Java 21+.

2.1 Prerequisites

- Docker Desktop (or Docker Engine + Compose plugin)
- Java 21+ (for the Quarkus server)
- Maven 3.9+ (for building the server and client plugin)
- Eclipse with Archi and the Archi modelling tool (for the client plugin)

2.2 Configuration

1. Copy `.env.example` to `.env`:

```
1 cp .env.example .env
```

Listing 1: Setup environment

2. Edit `.env` if you need to change the Neo4j password or Kafka bootstrap address. The defaults work for local development:

```
1 NEO4J_USER=neo4j
2 NEO4J_PASSWORD=devpassword
3 NEO4J_URI=bolt://localhost:7687
4 KAFKA_BOOTSTRAP_SERVERS=localhost:9092
5 KAFKA_TOPIC_PREFIX=archimesh.model
6 KAFKA_CONSUMER_GROUP_ID=archimesh-server
7 ARCHIMESH_CONSISTENCY_CHECK=false
8 ARCHIMESH_AUTHZ_ENABLED=false
9 ARCHIMESH_IDENTITY_MODE=bootstrap
10 ARCHIMESH_LOG_LEVEL=INFO
```

Listing 2: Default .env values

2.3 Starting infrastructure

Start Neo4j and Kafka with Docker Compose:

```
1 docker compose up -d
```

Listing 3: Start containers

Services available after startup:

Neo4j Browser `http://localhost:7474`

Neo4j Bolt `bolt://localhost:7687`

Kafka broker `localhost:9092`

Kafka UI `http://localhost:8080`

Apply Neo4j constraints and indexes:

2. GETTING STARTED

```
1 docker compose exec -T neo4j cypher-shell \  
2   -u "${NEO4J_USER:-neo4j}" \  
3   -p "${NEO4J_PASSWORD:-devpassword}" \  
4   < neo4j/schema.cypher
```

Listing 4: Apply schema

2.4 Building and running the server

The server is a Quarkus application in `server/`:

```
1 cd server  
2 mvn quarkus:dev
```

Listing 5: Build and run server

The server starts on port 8081. Verify:

```
1 curl -s http://localhost:8081/admin/overview | jq .
```

Listing 6: Health check

2.5 Creating a model

Models must be explicitly registered before use:

```
1 curl -s -X POST http://localhost:8081/admin/models/demo | jq .
```

Listing 7: Register model

2.6 Building the client plugin

The client plugin is an Eclipse RCP bundle:

```
1 cd client  
2 mvn -q -pl org.gautelis.archimesh.plugin -am install -DskipTests
```

Listing 8: Build client plugin

Install the built plugin into Archi using:

```
1 ./scripts/install-archimesh-plugin.sh
```

Listing 9: Install plugin

2.7 Building the admin UI

The admin UI is a Svelte application:

```
1 cd admin-web  
2 npm install  
3 npm run build
```

Listing 10: Build admin UI

2. GETTING STARTED

Or use the convenience script:

```
1 ./scripts/build-admin-web.sh
```

Listing 11: Build admin UI via script

The built static app is served by Quarkus at `/admin-ui/`.

2.8 First collaborative session

1. Start two Archi instances with the plugin installed.
2. In the first instance, create a new model or open an existing one.
3. Use **Tools > Connect Archimesh...** to connect. Enter the WebSocket URL (e.g. `ws://localhost:8081`) and model id (demo).
4. In the second instance, connect to the same model.
5. Make edits in either instance. Changes appear in the other within the broadcast latency of the Kafka fan-out path (typically under one second).

2.9 Validation scripts

The project includes local validation scripts for fast feedback:

```
1 # Fast hardening and parity checks (no Docker infra needed)
2 ./scripts/validate-local.sh fast
3
4 # Full client + server gate
5 ./scripts/validate-local.sh gate
6
7 # Full gate with local Kafka/Neo4j-backed integration tests
8 ./scripts/validate-local.sh infra
9
10 # Full gate with websocket end-to-end coverage
11 ./scripts/validate-local.sh ws
```

Listing 12: Validation commands

These are the supported release gate entrypoints. Run `fast` before committing; run `gate` or `infra` before pushing.

2.10 Stopping

```
1 docker compose down
```

Listing 13: Stop infrastructure

Add `-v` to also remove volumes (clean slate):

```
1 docker compose down -v
```

Listing 14: Stop and remove volumes

2.11 Where to go next

- Section 3 for the system architecture and component layout.
- Section 4 for the client plugin internals.
- Section 5 for the complete API reference.
- Section 10 for the admin UI and supervision workflows.
- `dev-setup.md` for additional local development tips.

3 Architecture

This chapter describes the overall system architecture, component layout and major data flows in Archimesh.

3.1 System purpose

The system lets multiple Archi clients collaborate on the same model timeline through a centralized server. It combines:

- a local Archi client plugin that captures and applies model changes
- a Quarkus server that validates, orders, persists and broadcasts changes
- Kafka for ordered model event fan-out
- Neo4j for the materialized model graph, op-log, tags and merge metadata

The model timeline is intentionally linear: one HEAD, no branches, immutable tags as named historical points.

3.2 High-level flow

1. A user edits a model in Archi.
2. The client plugin captures local EMF changes and converts them into Archimesh operations.
3. The server validates the op batch against preconditions, locks and idempotency state, then assigns authoritative revisions.
4. The server persists the batch, updates the materialized graph and broadcasts it via Kafka.
5. Other connected clients apply the accepted operations into their local Archi model.

3.3 Component layout

Component	Responsibility
server/	Quarkus server: WebSocket and REST endpoints, authorization, validation, revision assignment, idempotency, persistence, broadcast.
client/org.gautelis.archimesh.plugin/	Eclipse RCP plugin: WebSocket client, EMF change capture, operation mapping, remote apply, offline outbox, cache persistence.
client/client-tests/	Client plugin tests: CRDT merge, operation mapping, folder sync, outbox replay, notation inventory.
admin-web/	Svelte admin UI: model overview, catalog, versions, access control, sessions, audit.
schemas/	JSON schemas for common types, op batches, operations, lock events, presence and notation contracts.
neo4j/scripts/	Cypher schema for constraints and indexes, plus a documented graph model. Build, validation, development and operational scripts.

3. ARCHITECTURE

`docker-compose.yml` Local dev infrastructure: Neo4j + Kafka.

3.4 Server component map

The server is organized into distinct packages:

Package	Responsibility
<code>endpoint</code>	WebSocket endpoint (<code>ArchimeshEndpoint</code>), REST endpoints (<code>ModelStateEndpoint</code> , <code>AdminEndpoint</code> , <code>AdminUiEndpoint</code> , <code>ConsistencyEndpoint</code>).
<code>service</code>	Central coordination (<code>ArchimeshService</code>), plus service interfaces for validation, revision, locks, Neo4j repository, Kafka publish/consume, session registry and idempotency.
<code>service.impl</code>	Concrete implementations: in-memory services for validation, revision, locks and sessions; Neo4j repository with split support classes for op-log, materialized state, reads and compaction; Kafka publisher and consumer implementations.
<code>auth</code>	Authorization: identity modes, role normalization, policy decision point (PDP), authorization service and transport.
<code>model</code>	Admin data classes: status, catalog, sessions, ACLs, export/import, compaction, integrity, audit.
<code>wire</code>	Wire protocol envelopes (<code>ClientEnvelope</code> , <code>ServerEnvelope</code>) and message types for inbound (<code>Join</code> , <code>SubmitOps</code> , <code>AcquireLock</code> , <code>ReleaseLock</code> , <code>Presence</code>) and outbound (<code>CheckoutSnapshot</code> , <code>CheckoutDelta</code> , <code>OpsAccepted</code> , <code>OpsBroadcast</code> , <code>LockEvent</code> , <code>PresenceBroadcast</code> , <code>Error</code>).

3.5 Client plugin component map

Package	Responsibility
<code>ws</code>	WebSocket session manager, outbox handling, join/rejoin decisions, local submit tracking, inbound message dispatch.
<code>emf</code>	EMF change capture, operation mapping, remote op application, CRDT merge (LWW, OR-set, tombstones), model controller.
<code>startup</code>	Eclipse startup hooks, workbench lifecycle bridge, status line bridge.
<code>handlers</code>	Menu command handlers: connect, disconnect, open server model, switch active model, show last conflict, resynchronize.
<code>ui</code>	Connect dialog, server model catalog dialog, open server model dialog.
<code>notation</code>	Notation serializer and deserializer for Archi diagram fields.
<code>util</code>	Authentication hint helpers and lightweight JSON utilities.

3.6 Two kinds of state

The system maintains two kinds of state at all times:

Local working state Inside each Archi client. This is the EMF model the user sees and edits. It is not authoritative.

Authoritative shared state On the Archimesh server. The Neo4j graph is the source of truth. The op-log is the replayable history.

The client plugin synchronizes between these two states: capturing local changes, submitting them to the server, and applying remote changes into the local model.

3.7 Snapshot vs op-log

The server keeps both a materialized snapshot-like graph and an append-only op-log in Neo4j. Why both exist:

- The materialized graph makes joins, exports and admin inspection fast (no replay needed).
- The op-log preserves the authoritative committed history for rebuild, import and time-travel.

This is why export packages contain both the current snapshot and the full op batch history. The snapshot is derived from the op-log; the op-log is the ground truth.

3.8 Folder model

Model-tree folders are first-class synchronized state with strict rules:

- Root folders have stable synthetic ids (e.g. `folder:root-business`).
- User folders have immutable ids — rename is allowed, but identity is stable.
- Object placement in folders is explicit and persisted.
- Cross-root folder moves are rejected by validation.
- Non-empty folder delete is rejected.

This avoids drift between clients and preserves Archi model-tree structure.

3.9 Linear timeline

Every model has exactly one linear revision timeline with:

- One moving `HEAD`
- Zero or more immutable tags
- No branches

Clients submit batches against a `baseRevision`. The server assigns an authoritative contiguous revision range. If a batch contains N ops it consumes N revisions. Tags are read-only pointers to historical revisions, stored with a snapshot of the model at that point.

Only `HEAD` is writable. Tagged revisions are read-only for both reads and writes.

3.10 Deployment and operational considerations

3.10.1 Transport security

The WebSocket and REST endpoints do not terminate TLS themselves. In production, deploy behind a reverse proxy (Nginx, Caddy, Traefik) that terminates TLS and forwards to the Quarkus server. Example configurations are provided in `server/examples/`.

When using `app.identity.mode=proxy`, the reverse proxy must also forward trusted identity headers, and the WebSocket upgrade path must carry those headers through the proxy to the server.

3.10.2 Reconnection backpressure

After a server restart, all previously connected clients may attempt to reconnect simultaneously. To avoid overwhelming the server:

- Clients should apply jitter to reconnection attempts (randomized delay 1–5 seconds before the first retry).
- The server should reject joins that arrive faster than a configurable rate limit.
- Admin diagnostics expose active session counts so operators can detect reconnection storms.

The current client implementation does not yet include reconnect jitter. This is a known operational gap for production deployments with many concurrent clients.

3.10.3 Kafka single-partition requirement

Each model's ops topic must have exactly one partition to preserve total order. If multiple partitions are used, the total order guarantee breaks and clients can receive operations in different orders, producing divergent state. This is a hard constraint, not a recommendation.

If Kafka publish fails, the server falls back to broadcasting `OpsBroadcast` directly to connected WebSocket sessions, ensuring clients on the same server instance still receive updates.

4 Client Plugin

The client plugin is an Eclipse RCP bundle that runs inside Archi. It captures local EMF model changes, submits them to the Archimesh server, and applies remote operations into the local model.

4.1 Plugin entry point

The plugin activator is

`org.gautelis.archimesh.plugin.ArchimeshPlugin`, an `AbstractUIPlugin` subclass.

On startup it creates the three central objects:

ArchimeshSessionManager WebSocket lifecycle, outbox handling, join/rejoin decisions and local submit tracking.

WorkbenchModelLifecycleBridge Hooks into Archi's workbench to detect model open, close, save and dirty-state changes.

ArchimeshStatusLineBridge Displays connection state, conflict banners and outbox capacity warnings in the Archi status line.

The plugin id is `org.gautelis.archimesh.plugin`. Debug and trace logging are controlled by the system property `archimesh.debug` and `archimesh.trace`, or equivalently the environment variables `ARCHIMESH_DEBUG` and `ARCHIMESH_TRACE`.

4.2 Session manager

`ArchimeshSessionManager` is the central client-side coordinator. It manages:

- WebSocket connection lifecycle (connect, disconnect, reconnect)
- Per-model persistent bounded outbox (FIFO queue with replay)
- Join/rejoin decision logic with cache metadata
- Echo suppression for locally submitted op batches
- Cache autosave policy (debounced saves on sync traffic)

4.2.1 Connection flow

1. The user selects **Tools > Connect Archimesh...** and provides the server URL, model id and credentials.
2. The session manager resolves the join decision:
 - If the session is server-backed, cache metadata is read from `~/Archi/archimesh-cache/<modelId>.meta.properties`.
 - If metadata is valid and the cached revision is known, the join includes `lastSeenRevision` so the server can send a delta.
 - If metadata is missing, mismatched or the revision is unknown, the join omits `lastSeenRevision` to request a cold snapshot.
3. The session manager opens a WebSocket to `<baseUrl>/models/<modelId>/stream` and sends a `Join` message.

4. The server responds with `CheckoutSnapshot` (full model state) or `CheckoutDelta` (incremental changes since `lastSeenRevision`).
5. The client applies the checkout payload into the local Archi model.

4.2.2 Rejoin and cache verification

On reconnect the session manager reads the cache metadata file. A cache projection is discarded (forcing a cold snapshot) if:

- The metadata file does not exist or cannot be read.
- The metadata `serverBacked` flag does not match.
- The `modelId` or `modelRef` does not match the target.
- The cache file is missing from disk.
- The cached revision is unknown (negative).
- The cache file exists but is implausibly small for a model with revision ≥ 1 (under 1 KB — likely truncated or corrupted).

After join, the server's first revision hint is compared against the cache revision used at join. If the server revision is behind the cache revision, the client requests a cold snapshot rebuild.

4.3 EMF change capture

`EmfChangeCapture` observes local EMF model changes via Archi's `ECore` notification mechanism. When the user edits the model, the capture layer:

1. Receives EMF notifications for element, relationship, view, folder and property changes.
2. Batches correlated changes (e.g. a view object create + its initial geometry update) into a single logical operation.
3. Defers connection-creation submission until all referenced identifiers (source, target, relationship, view) are available, then emits the relationship and connection in a single `SubmitOps` batch.
4. Passes the collected operations to `OpMapper` for wire-format encoding.

The capture layer runs under a remote-apply guard so that operations applied from the server are not re-captured and re-submitted as local edits.

4.4 Operation mapping

`OpMapper` converts captured EMF changes into Archimesh wire-format operations. The supported operation types are:

Operation	Purpose
<code>CreateElement</code>	Create a new ArchiMate element.
<code>UpdateElement</code>	Update element name or documentation.
<code>DeleteElement</code>	Delete an element and cascade-delete its diagram representations.
<code>CreateRelationship</code>	Create a relationship between two elements.

<code>UpdateRelationship</code>	Update relationship fields.
<code>DeleteRelationship</code>	Delete a relationship and cascade-delete its connections.
<code>CreateFolder</code>	Create a user folder.
<code>UpdateFolder</code>	Rename a folder or update its documentation.
<code>DeleteFolder</code>	Delete an empty user folder.
<code>CreateView</code>	Create a new diagram view.
<code>UpdateView</code>	Update view metadata.
<code>DeleteView</code>	Delete a view and its contents.
<code>CreateViewObject</code>	Place an element on a diagram.
<code>UpdateViewObjectOpaque</code>	Update geometry, style or label fields of a diagram object.
<code>DeleteViewObject</code>	Remove a diagram object.
<code>CreateConnection</code>	Draw a connection between two diagram objects.
<code>UpdateConnectionOpaque</code>	Update connection style, label or bendpoint fields.
<code>DeleteConnection</code>	Remove a diagram connection.
<code>AddElementToFolder</code>	Place an element in a folder.
<code>RemoveElementFromFolder</code>	Remove an element from a folder.
<code>SetProperty</code>	Set a key-value property on a model entity.
<code>UnsetProperty</code>	Remove a property.
<code>AddPropertySetMember</code>	Add an element to a property set.
<code>RemovePropertySetMember</code>	Remove an element from a property set.
<code>AddViewObjectChildMember</code>	Nest a view object inside another.
<code>RemoveViewObjectChildMember</code>	Remove nested view object membership.

Each operation carries a causal object with `clientId`, `lamport` and `opId` for LWW merge semantics.

4.5 Remote operation application

`RemoteOpApplier` applies snapshots and remote broadcasts into the local EMF model. Key behaviors:

- Applies operations under a remote-apply guard to prevent re-capture.
- Applies per-field LWW merges for notation fields (geometry, style, labels) using causal clock comparison.
- Maintains element field clocks and tombstones for delete safety.
- Handles property-level LWW and OR-set semantics for `SetProperty`, `UnsetProperty`, `AddPropertySetMember`, `RemovePropertySetMember`.
- Restores folder structure before folder memberships during snapshot application.
- Restores views before view objects and connections.
- Defers dependency-sensitive operations when referenced entities are not yet created.

4.6 Offline outbox

When the WebSocket is unavailable or a live send fails, the client queues operations in a persistent bounded outbox. Key guarantees:

- **FIFO per model.** Operations are drained in submission order.
- **Rebase before send.** Each replayed op has its `baseRevision` updated to the current known server revision.

- **Ack-gated removal.** Entries are removed only after the server sends a matching `OpsAccepted`. Websocket send completion is not enough — the server must confirm commit.
- **Deterministic retry.** Failed sends are retried with capped exponential backoff. Entries failing after the maximum replay attempts (5) are dropped as poison entries.
- **Conflict resolution.** Entries receiving `PRECONDITION_FAILED` or `LOCK_CONFLICT` from the server are deterministically dropped. A conflict callback is fired, surfaced in the UI status line.
- **Bounded queue.** Maximum 1000 entries. When full, the oldest entry is dropped. A warning fires at 80% capacity (800 entries) and is displayed in the status line.
- **Durable persistence.** The outbox is persisted to `~/Archi/archimesh-cache/<modelId>.archimate.outbox.properties` and reloaded on reconnect.

4.7 Cache policy

When running in server-backed mode, the client treats the local `.archimate` file as a cache projection, not the primary artifact.

Cache path `~/Archi/archimesh-cache/<modelId>.archimate`

Metadata file `<cacheFile>.meta.properties` stores `modelId`, `modelRef`, `lastKnownRevision`, `serverBacked` and `savedAtEpochMs`.

Autosave triggers Debounced saves (2 s cooldown) after `CheckoutSnapshot`, `CheckoutDelta`, `OpsBroadcast` and `OpsAccepted`.

Force-save triggers On model attach, detach and disconnect.

Discard policy Cache is discarded if inconsistent, invalid or not server-backed at reconnect.

4.8 Echo suppression

To avoid double-applying their own submitted operations, clients track the most recent 256 locally submitted `opBatchId` values. When an `OpsBroadcast` arrives via the Kafka fan-out path, the client checks whether the batch was locally submitted. If so, the broadcast is silently ignored. The local model was already updated optimistically and will be corrected by the `OpsAccepted` confirmation if needed.

4.9 Menu commands

The plugin contributes the following commands to Archi's **Tools** menu:

Connect Archimesh... Connect the active model to an Archimesh server. Opens a dialog for server URL, model id, user, session id and optional bearer token.

Disconnect Archimesh Disconnect from the server, save cache and return to local-only mode.

Open Archimesh Model from Server... Browse the server model catalog and open a model from the server.

Switch Active Model to Server-Backed... Attach Archimesh to the currently active local model, making it server-backed.

Show Last Archimesh Conflict Display details for the most recent conflict (precondition failure or lock conflict).

Resynchronize Model Discard the local projection and pull a fresh snapshot from the server.

5 Server API

This chapter documents the WebSocket and REST API endpoints exposed by the Archimesh server.

5.1 WebSocket endpoint

Path /models/{modelId}/stream

Protocol WebSocket (RFC 6455)

Sub-protocol None (plain JSON text frames)

The WebSocket endpoint handles real-time collaboration. Each connection is scoped to one model.

5.1.1 WebSocket lifecycle

1. Client opens a WebSocket connection to the endpoint.
2. Server performs authorization checks (see Section 9) and either accepts the connection or closes it with policy violation status.
3. Client sends a `Join` message to request model state.
4. Server responds with `CheckoutSnapshot` or `CheckoutDelta`.
5. Client sends `SubmitOps`, `AcquireLock`, `ReleaseLock` and `Presence` messages during the session.
6. Server broadcasts `OpsAccepted`, `OpsBroadcast`, `LockEvent`, `PresenceBroadcast` and `Error` messages to connected sessions.
7. Client or server closes the WebSocket. On close the server unregisters the session and releases held locks.

5.1.2 Inbound messages (client to server)

All inbound messages are wrapped in a `ClientEnvelope`:

```
1 {  
2   "type": "<MessageType>",  
3   "payload": { ... }  
4 }
```

Listing 15: ClientEnvelope shape

Type	Purpose	Key payload fields
Join	Request model checkout	lastSeenRevision (optional), ref (optional; HEAD or tag name), actor (userId, sessionId)
SubmitOps	Submit one batch of operations	baseRevision, opBatchId, actor, ops (array)
AcquireLock	Acquire a notation lease	actor, targets (array of entity ids), ttlMs
ReleaseLock	Release a notation lease	actor, targets (array of entity ids)

5. SERVER API

Presence	Broadcast cursor/selection	actor, viewId, selection (array), cursor (object)
----------	----------------------------	---

5.1.3 Join and checkout

When the server receives a Join:

1. The model must exist (registered via admin API).
2. Authorization is checked (join action).
3. The requested ref is resolved (HEAD or tag name).
4. If `lastSeenRevision` is present and a delta is available (the revision gap is within the op-log window), the server responds with `CheckoutDelta` containing the missing operations.
5. Otherwise, the server responds with `CheckoutSnapshot` containing the full materialized model state at the requested ref.

5.1.4 SubmitOps processing

The `SubmitOps` pipeline (implemented in `ArchimeshService`):

1. Model registration and writable ref checks.
2. Validation of the op batch structure.
3. Normalization and expansion (bootstrap view ref normalization, cascade delete expansion).
4. Revision-ahead check: if `baseRevision > headRevision`, broadcast `Error (REVISION_AHEAD)` to all sessions.
5. Precondition validation: all entity references must exist either in persisted state or in earlier ops within the same batch.
6. Idempotency check: if the `opBatchId` has been seen before, broadcast `OpsAccepted` with the known revision range.
7. Lock conflict check: for operations requiring notation locks (`UpdateViewObjectOpaque`, `UpdateConnecti`), verify no other session holds a conflicting lock.
8. Revision range assignment: allocate `from..to` consuming one revision per op.
9. Causal metadata normalization: stamp each op with `clientId`, `lamport`, `opId`.
10. Idempotency recording.
11. Persistence: append to op-log, apply to materialized state, update head revision.
12. Optional post-commit consistency checks.
13. Kafka publish (with fallback direct broadcast on failure).
14. Broadcast `OpsAccepted` to all sessions.

If any step fails before persistence, an `Error` message is broadcast with a specific code (`REVISION_AHEAD`, `PRECONDITION_FAILED`, `LOCK_CONFLICT`).

5.1.5 Outbound messages (server to client)

Type	Purpose	Key payload fields
------	---------	--------------------

5. SERVER API

CheckoutSnapshot	Full model state	modelId, headRevision, elements, relationships, views, folders, etc.
CheckoutDelta	Incremental changes	modelId, fromRevision, toRevision, ops (array)
OpsAccepted	Batch commit confirmation	opBatchId, baseRevision, assignedRevisionRange
OpsBroadcast	Remote batch broadcast	opBatch (full committed batch)
LockEvent	Lock state change	action (acquired/released/expired), actor, targets
PresenceBroadcast	Remote cursor/selection	actor, viewId, selection, cursor
Error	Rejection or failure	code, message

5.2 REST API

The server exposes REST endpoints under the base path /.

5.2.1 Model state endpoints

Endpoint	Method	Purpose
/models/{modelId}/snapshot	GET	Get the current materialized snapshot of a model. Optional query parameter <code>?ref=<tagName></code> .
/models/{modelId}/rebuild	POST	Rebuild materialized state from the op-log. Clears materialized nodes and replays all commits.

Example:

```
1 curl -s http://localhost:8081/models/demo/snapshot | jq .
```

Listing 16: Get snapshot

5.2.2 Admin endpoints

Endpoint	Method	Purpose
/admin/models	GET	List registered models (catalog).
/admin/models/{modelId}	POST	Register a new model.
	DELETE	Delete a model. Query param <code>?force=true</code> to override active-session guard.
/admin/models/{modelId}/status	GET	Aggregated status: snapshot counts, revision, consistency.
/admin/models/{modelId}/rebuild-and-status	POST	Rebuild and return status in one call.

<code>/admin/models/{modelId}/window?limit=25</code>	GET	Model window: status + recent activity + recent op batches.
<code>/admin/overview?limit=25</code>	GET	Overview for all known active/recent models.
<code>/admin/models/{modelId}/integrity</code>	GET	Integrity report: missing references, orphan nodes.
<code>/admin/models/{modelId}/acl</code>	GET, PUT	Read or update per-model ACL (admin, writer, reader user lists).
<code>/admin/models/{modelId}/tag</code>	POST	Create an immutable tag at the current HEAD.
<code>/admin/models/{modelId}/export</code>	GET	Export model package (snapshot + op-log + tags + ACL).
<code>/admin/models/import?overwrite=true false</code>	POST	Import a model package.
<code>/admin/models/{modelId}/compact</code>	POST	Run compaction on the model (prune old op-log entries and merge metadata).
<code>/admin/auth/diagnostics</code>	GET	Authorization diagnostics: resolved identity, subject, normalized roles.
<code>/admin/sessions</code>	GET	Active WebSocket sessions across all models.
<code>/admin/audit/config</code>	GET	Current audit configuration.

Examples:

```
1 # List all models
2 curl -s http://localhost:8081/admin/models | jq .
3
4 # Create a model
5 curl -s -X POST http://localhost:8081/admin/models/demo | jq .
6
7 # Get model status
8 curl -s http://localhost:8081/admin/models/demo/status | jq .
9
10 # Get model window
11 curl -s http://localhost:8081/admin/models/demo/window?limit=25 | jq .
12
13 # Get overview
14 curl -s http://localhost:8081/admin/overview?limit=25 | jq .
15
16 # Delete model
17 curl -s -X DELETE http://localhost:8081/admin/models/demo | jq .
18
19 # Force delete (override active-session guard)
20 curl -s -X DELETE "http://localhost:8081/admin/models/demo?force=true" | jq .
```

Listing 17: Admin API examples

5.2.3 Admin UI endpoint

Path `/admin-ui/`

Serves Static Svelte admin application (built from `admin-web/`).

When running in dev mode, the admin UI is served from `server/src/main/resources/META-`

`INF/resources/admin-ui/` (populated by the build script). In production, the Quarkus application serves the built static assets.

6 Persistence

The Archimesh server persists all model state to Neo4j. This chapter describes the graph model, repository layer and key invariants.

6.1 Repository layer

The persistence layer is organized into a split repository with a main coordinator and specialized support classes:

Neo4jRepositoryImpl Main coordinating repository. Orchestrates op-log writes, materialized state mutations, head revision updates, snapshots, rebuilds, imports, exports, compactions and integrity checks.

Neo4jOpLogSupport Commit and operation persistence. Writes `:Commit` and `:Op` nodes, maintains the NEXT chain.

Neo4jMaterializedStateSupport Materialized graph mutation. Creates, updates and deletes elements, relationships, views, view objects, connections and folders. Applies LWW merges and tombstone checks. Maintains property clocks.

Neo4jReadSupport Snapshot export, checkout delta reads, consistency checks. Reads materialized state and converts it to the JSON snapshot/export format.

Neo4jCompactionSupport Metadata compaction. Prunes old commits and property clocks below the retention watermark. Reports eligibility of tombstones and field clocks for future compaction.

6.2 Graph model

The database stores four kinds of state per model:

1. Model catalog and ACL metadata
2. Materialized current state
3. Committed op-log history
4. Merge metadata (tombstones, property clocks)

All identity is model-scoped: materialized entity identity is `(modelId, id)`; commit identity is `(modelId, opBatchId)`; tag identity is `(modelId, tagName)`.

6.2.1 Core model node

Every model is rooted at a `:Model` node:

```
1 {  
2   modelId, modelName,  
3   registered: true,  
4   headRevision: <number>,  
5   createdAt, updatedAt,  
6   adminUsers: ["user1", ...],  
7   writerUsers: ["user2", ...],  
8   readerUsers: ["user3", ...]  
9 }
```

Listing 18: Model node properties

ACL data is stored directly on `:Model` as user-id arrays.

6.2.2 Materialized state nodes

Label	Properties
<code>:Element</code>	<code>modelId</code> , <code>id</code> , <code>archimateType</code> , <code>name</code> , <code>documentation</code> , <code>name_lamport</code> , <code>name_clientId</code> , <code>documentation_lamport</code> , <code>documentation_clientId</code>
<code>:Relationship</code>	<code>modelId</code> , <code>id</code> , <code>archimateType</code> , <code>name</code> , <code>documentation</code> , <code>sourceId</code> , <code>targetId</code> , plus LWW clocks for <code>name</code> , <code>documentation</code> , <code>source</code> and <code>target</code> .
<code>:Folder</code>	<code>modelId</code> , <code>id</code> , <code>folderType</code> , <code>name</code> , <code>documentation</code> , plus LWW clocks.
<code>:View</code>	<code>modelId</code> , <code>id</code> , <code>name</code> , <code>documentation</code> , <code>notationJson</code> , plus LWW clocks.
<code>:ViewObject</code>	<code>modelId</code> , <code>id</code> , <code>notationJson</code> , plus per-field LWW clocks for <code>geometry</code> , <code>style</code> and <code>label</code> fields.
<code>:Connection</code>	<code>modelId</code> , <code>id</code> , <code>sourceViewObjectId</code> , <code>targetViewObjectId</code> , <code>representsId</code> , <code>notationJson</code> , plus per-field LWW clocks.

6.2.3 Materialized relationships

```
1 (:Model)-[:HAS_ELEMENT]->(:Element)
2 (:Model)-[:HAS_REL]->(:Relationship)
3 (:Relationship)-[:SOURCE]->(:Element)
4 (:Relationship)-[:TARGET]->(:Element)
5
6 (:Model)-[:HAS_FOLDER]->(:Folder)
7 (:Folder)-[:HAS_FOLDER]->(:Folder)
8 (:Folder)-[:CONTAINS_ELEMENT]->(:Element)
9 (:Folder)-[:CONTAINS_REL]->(:Relationship)
10 (:Folder)-[:CONTAINS_VIEW]->(:View)
11
12 (:Model)-[:HAS_VIEW]->(:View)
13 (:View)-[:CONTAINS]->(:ViewObject)
14 (:ViewObject)-[:REPRESENTS]->(:Element)
15 (:ViewObject)-[:CHILD_MEMBER]->(:ViewObject)
16
17 (:View)-[:CONTAINS]->(:Connection)
18 (:Connection)-[:REPRESENTS]->(:Relationship)
19 (:Connection)-[:FROM]->(:ViewObject)
20 (:Connection)-[:TO]->(:ViewObject)
```

Listing 19: Core relationships

6.2.4 Folder rules

- Root folders use stable synthetic ids: `folder:root-strategy`, `folder:root-business`, `folder:root-diagrams`, etc.
- User folders use immutable ids — rename changes the `name` property but not the node identity.
- A model-tree object belongs to exactly one folder at a time.
- Cross-root folder moves are rejected by validation.
- Non-empty folder delete is rejected.

6.2.5 Op-log history

Commit nodes and operation nodes form the append-only operation log:

```
1 (:Commit {  
2   modelId, opBatchId,  
3   revisionFrom, revisionTo,  
4   userId, sessionId, ts  
5 })  
6  
7 (:Op {  
8   seq, type, targetId, payloadJson  
9 })
```

Listing 20: Op-log nodes

Relationships:

```
1 (:Model)-[:HAS_COMMIT]->(:Commit)  
2 (:Commit)-[:HAS_OP]->(:Op)  
3 (:Commit)-[:NEXT]->(:Commit)
```

Listing 21: Op-log relationships

- `revisionFrom..revisionTo` is the assigned revision range for one committed submit batch.
- `payloadJson` stores the normalized op payload.
- `seq` preserves op order inside a batch.
- `NEXT` keeps the timeline linear.

6.2.6 Tags

Tags are stored as standalone nodes:

```
1 (:ModelTag {  
2   modelId, tagName, description,  
3   revision, createdAt, snapshotJson  
4 })
```

Listing 22: Tag node

- Tags are immutable — once created, they cannot be modified or deleted (tag deletion is disabled by default).
- `snapshotJson` stores the materialized snapshot at the tagged revision.
- Tags do not create branches; they are read-only pointers on the linear timeline.

6.2.7 Merge metadata — tombstones

Tombstones prevent stale recreates after deletes:

```
1 (:ElementTombstone { modelId, id, lamport, clientId })  
2 (:RelationshipTombstone { modelId, id, lamport, clientId })  
3 (:ViewTombstone { modelId, id, lamport, clientId })  
4 (:ViewObjectTombstone { modelId, id, lamport, clientId })  
5 (:ConnectionTombstone { modelId, id, lamport, clientId })
```

Listing 23: Tombstone nodes

Tombstones are anchored from `:Model` via specific relationship types (`HAS_ELEMENT_TOMBSTONE`, etc.). Compaction reports tombstone eligibility but retains them for delete safety.

6.2.8 Merge metadata — property clocks

Property merge state is stored in dedicated metadata nodes:

```
1 (:PropertyClock {
2   modelId, ownerId, key,
3   lamport, clientId, deleted
4 })
```

Listing 24: PropertyClock node

Anchored via `(:Model) -[:HAS_PROPERTY_CLOCK]->(:PropertyClock)`. Supports LWW and OR-set style property semantics. Compaction may prune old property clocks below the retention watermark.

6.3 Neo4j constraints

The schema file at `neo4j/schema.cypher` defines the following uniqueness constraints:

```
1 (:Model {modelId}) UNIQUE
2 (:Element {modelId, id}) UNIQUE
3 (:Relationship {modelId, id}) UNIQUE
4 (:View {modelId, id}) UNIQUE
5 (:Folder {modelId, id}) UNIQUE
6 (:ViewObject {modelId, id}) UNIQUE
7 (:Connection {modelId, id}) UNIQUE
8 (:Commit {modelId, opBatchId}) UNIQUE
9 (:ModelTag {modelId, tagName}) UNIQUE
```

Listing 25: Uniqueness constraints

An index on `(:Commit)` by `(modelId, revisionFrom, revisionTo)` supports efficient delta window queries.

6.4 Persistence invariants

- Materialized entity identity is `(modelId, id)` — never rely on node-internal ids across queries.
- Commit/idempotency identity is `(modelId, opBatchId)` — duplicate submission of the same batch is a no-op.
- Repository write paths must fail fast on Neo4j persistence errors — never silently skip a mutation.
- Notation field definitions must flow through `NotationMetadata`, the shared source of truth for supported notation fields and persisted merge metadata.
- Compaction never allows stale `Create*` or `Update*` to beat a logically newer delete — tombstones are retained.

6.5 Configuration

6. PERSISTENCE

Property	Env var	Default
app.neo4j.uri	NEO4J_URI	bolt://localhost:7687
app.neo4j.username	NEO4J_USER	neo4j
app.neo4j.password	NEO4J_PASSWORD	devpassword

In production, always set `NEO4J_PASSWORD` to a strong value. The default `devpassword` is for local development only.

7 Messaging

Kafka is used as the ordered fan-out path for accepted Archimesh operation batches, lock events and presence events.

7.1 Topic layout

Three topic kinds exist per model:

Topic pattern	Content	Partitioning
<code>archimesh.model.<modelId>.ops</code>	Commited op batches	Exactly 1 partition (required)
<code>archimesh.model.<modelId>.locks</code>	Lock acquire/release/expire	1 partition
<code>archimesh.model.<modelId>.presence</code>	Cursor/session broadcasts	1 partition (or keyed by sessionId)

The topic prefix is configurable via `KAFKA_TOPIC_PREFIX` (default `archimesh.model`).

7.2 Single-partition requirement

Each model's ops topic **must** have exactly one partition. This is a hard constraint, not a recommendation. Multiple partitions would allow operations to be consumed in different orders by different clients, breaking the total order guarantee that the deterministic merge semantics depend on.

Topic keys:

- ops: constant key = `<modelId>`
- locks: constant key = `<modelId>`
- presence: key = `<sessionId>` (or `<userId>`)

7.3 Producer

The `KafkaPublisherImpl` uses a `KafkaProducer<String, String>` with:

- `acks=all` — wait for all in-sync replicas
- `retries=3`
- `delivery.timeout.ms=120000`
- String serializers for both key and value

Publishing is asynchronous. The `publishOps` method returns a `CompletableFuture<Void>` that completes when the broker acknowledges the send. If the send fails, the server falls back to broadcasting `OpsBroadcast` directly to connected `WebSocket` sessions.

7.4 Consumer

The `KafkaConsumerImpl` runs a single-threaded consume loop with:

- Group id: `archimesh-server` (configurable via `KAFKA_CONSUMER_GROUP_ID`)
- Auto offset reset: `earliest`
- Topic subscription via regex pattern matching all `<prefix>.<modelId>.<kind>` topics
- 500 ms poll timeout

On receiving a record, the consumer parses the topic name to extract the model id and kind, then broadcasts the payload to all sessions registered for that model:

ops Broadcasts `OpsBroadcast` to all sessions for the model. The submitter suppresses its own echo.

locks Broadcasts `LockEvent`.

presence Broadcasts `PresenceBroadcast`.

The consumer is resilient to Kafka unavailability: on connection failure it sleeps and retries. While the consumer is down, the direct broadcast fallback in the submit path (Section 5) ensures connected clients still receive updates.

7.5 Retention

ops Long retention if Kafka is used for rebuild, otherwise moderate. The Neo4j op-log is the durable log; Kafka is the fan-out path.

locks Short retention (minutes). Lock state is ephemeral.

presence Very short retention (minutes). Presence is transient.

7.6 Fallback broadcast

If Kafka publish fails (broker unavailable, topic error, serialization failure), the `ArchimeshService` catches the exception and broadcasts `OpsBroadcast` directly to all sessions registered for the model. This ensures that connected clients on the same server instance still receive remote updates even when the Kafka fan-out path is unavailable.

The fallback uses the same `OpsBroadcastMessage` envelope as the consumer path, so clients process it identically. Echo suppression on the submitting client prevents double-application.

7.7 Configuration

Property	Env var	Default
<code>kafka.bootstrap.servers</code>	<code>KAFKA_BOOTSTRAP_SERVERS</code>	<code>localhost:9092</code>
<code>kafka.topic.prefix</code>	<code>KAFKA_TOPIC_PREFIX</code>	<code>archimesh.model</code>
<code>kafka.consumer.group-id</code>	<code>KAFKA_CONSUMER_GROUP_ID</code>	<code>archimesh-server</code>
<code>kafka.consumer.auto-offset-reset</code>	<code>KAFKA_CONSUMER_AUTO_OFFSET_RESET</code>	<code>earliest</code>

8 CRDT and Merge Semantics

Archimesh uses a hybrid consistency model: the server is the single point of serialization for revision ordering, but per-field CRDT merges handle concurrent edits on different fields of the same object without requiring explicit locks.

8.1 Causal identity

Every operation carries a `causal` object:

```
1 {  
2   "clientId": "<stable client/session id>",  
3   "lamport": <monotonic per-client counter>,  
4   "opId": "<idempotency identity>"  
5 }
```

Listing 26: Causal metadata

If the client omits `causal`, the server fills it deterministically from the assigned revision range. The comparison rule for Last-Writer-Wins:

$(\text{lamportA}, \text{clientIdA})$ wins over $(\text{lamportB}, \text{clientIdB})$ iff $\text{lamportA} > \text{lamportB}$, or $\text{lamportA} == \text{lamportB}$ and $\text{clientIdA} > \text{clientIdB}$ (lexicographic).

8.2 Merge types by domain

8.2.1 Semantic element fields

Fields: `name`, `documentation`.

- Type: LWW Register per field per element.
- Metadata: field clock map `elementId -> { field -> (lamport, clientId) }`.
- Merge: apply incoming field if the incoming tuple wins the field clock; otherwise ignore.

8.2.2 Relationship fields

Fields: `name`, `documentation`, `sourceId`, `targetId`.

Same LWW register semantics as elements, with per-field clocks stored on the `:Relationship` node.

8.2.3 Diagram notation fields

View objects and connections use per-field LWW registers for:

View objects `x`, `y`, `width`, `height`, `type`, `alpha`, `lineAlpha`, `lineWidth`, `lineStyle`, `textAlignment`, `textPosition`, `gradient`, `iconVisibleState`, `deriveElementLineColor`, `fillColor`, `lineColor`, `font`, `fontColor`, `iconColor`, `imagePath`, `imagePosition`, `name`, `documentation`.

Connections type, nameVisible, textAlignment, textPosition, lineWidth, name, lineColor, font, fontColor, documentation, bendpoints.

Per-field notation clocks are stored as properties on the `:ViewObject` and `:Connection` nodes (e.g. `notation_lamport_x`, `notation_clientId_x`).

8.2.4 View object child membership

Type: OR-set. Operations: `AddViewObjectChildMember`, `RemoveViewObjectChildMember`.

Member clocks are stored via `:PropertyClock` nodes in the `VIEWOBJECT_CHILD` namespace. Materialized as `(:ViewObject) - [:CHILD_MEMBER] -> (:ViewObject)` edges.

8.2.5 Property set membership

Type: OR-set. Operations: `AddPropertySetMember`, `RemovePropertySetMember`.

Member clocks are stored via `:PropertyClock` nodes. Materialized as canonical JSON string arrays in the snapshot/export output.

8.3 Delete semantics — tombstones

When an entity is deleted, a tombstone is recorded with the deleting operation's causal clock:

```
1 { modelId, id, lamport, clientId }
```

Listing 27: Tombstone record

Rules:

- `Delete*` always records or updates the tombstone (even if the entity was already missing).
- `Stale Update*` or `Create*` (with a causal clock older than the tombstone) is ignored — preventing resurrection.
- A newer `Create*` (with a causal clock that beats the tombstone) may clear the tombstone and recreate the entity.

Tombstones persist across compactions. They are *reported* as eligible under the watermark but *retained* for delete safety.

8.4 Notation field validation

The server validates notation field keys against a whitelist defined in `NotationMetadata`. Operations containing unknown notation keys are rejected with `PRECONDITION_FAILED`. This prevents clients from emitting fields the server cannot merge deterministically.

The whitelist covers all currently supported notation fields for view objects and connections. The notation inventory is kept in parity across:

- Server runtime whitelist constants (`ArchimeshService`).

- JSON schema definitions (`schemas/ops.json`).
- Client serializer/deserializer field coverage.
- Client inventory test expectations (`OpMapperNotationInventoryTest`).

The script `scripts/check-notation-parity.sh` verifies this parity and is wired into the preflight hardening checks.

8.5 Lock model

Lease-based locks are used for notation-sensitive edits:

- Operations requiring locks: `UpdateViewObjectOpaque`, `UpdateConnectionOpaque`.
- Lock targets are the affected view object or connection ids.
- Default TTL: 10 seconds.
- A client must hold the lock for a target before submitting an opaque notation update for that target.
- If another client holds a conflicting lock, the submit is rejected with `LOCK_CONFLICT`.
- Locks are automatically released on `WebSocket` close.
- Locks are advisory, not enforced by the merge layer — they reduce destructive interleaving on noisy diagram edits while keeping semantic collaboration workable.

8.6 CRDT maturity

The CRDT implementation covers the currently supported operation set at the P0 and P1 levels described in `crdt-spec.md`. Remaining work includes:

- Extending collection-aware operations to additional domains beyond property sets and view object children.
- Gathering long-running workload evidence that metadata growth remains operationally bounded under production-like usage.

Any remaining non-CRDT domains are explicitly marked as unsupported rather than left as implicit gaps.

9 Authorization

Authorization is enforced in Quarkus using a server-side Policy Enforcement Point (PEP) and a project-specific in-process Policy Decision Point (PDP).

9.1 Identity modes

Identity resolution is pluggable. The mode is selected via `ARCHIMESH_IDENTITY_MODE` (or `app.identity.mode`).

bootstrap Local/dev mode. No external identity provider.

- REST: identity from `X-Archimesh-User` and `X-Archimesh-Roles` headers.
- WebSocket: identity from query parameters `user` and `roles`.
- Intended for local development and simple standalone deployments.

proxy Trusted forwarded headers from a reverse proxy.

- REST: identity from trusted forwarded headers (default `X-Forwarded-User`, `X-Forwarded-Roles`).
- WebSocket: identity from trusted forwarded headers captured during the handshake.
- Header names are configurable via `app.identity.proxy.user-header` and `app.identity.proxy.roles-header`.
- Do not expose `proxy` mode directly to untrusted clients without a trusted proxy in front.

oidc Direct principal/role resolution through Quarkus auth.

- REST: identity from the authenticated Quarkus `SecurityContext`.
- WebSocket: identity from the authenticated WebSocket upgrade principal and role checks captured during the handshake.
- Can use local JWT verification or an external OIDC provider.
- No reverse proxy required in this mode.

9.2 Role model

Role names from the identity provider are normalized into three canonical internal roles:

admin Catalog-wide administration: register/delete models, manage ACLs, run compactions, export/import, view diagnostics.

model_reader Read model snapshots and join read-only sessions.

model_writer Submit operations, acquire locks, send presence. Implies `model_reader`.

Role names are configurable:

```
1 # Default canonical role names
2 app.authz.admin-role=admin
3 app.authz.reader-role=model_reader
4 app.authz.writer-role=model_writer
5
6 # Optional provider-specific aliases (comma-separated)
7 app.authz.admin-role-aliases=realm-admin,archimesh-admin
8 app.authz.reader-role-aliases=realm-viewer,archimesh-reader
9 app.authz.writer-role-aliases=realm-editor,archimesh-writer
```

Listing 28: Role configuration

Aliases normalize provider-specific group names (e.g. Keycloak realm roles, Auth0 permissions) into the canonical internal roles before the PDP evaluates policy.

9.3 Policy model

The PDP decides on these actions:

- Admin catalog actions (list models, create, delete)
- Model join
- Snapshot read
- Submit operations
- Lock acquire/release
- Presence broadcast
- Tag creation
- ACL management
- Export/import
- Compaction

Authorization is deny-by-default when `ARCHIMESH_AUTHZ_ENABLED=true`.

9.3.1 Model-scoped ACLs

Per-model ACLs are stored on the `:Model` node as `adminUsers`, `writerUsers` and `readerUsers` arrays. When an ACL is configured for a model, it overrides the generic reader/writer role checks for that model:

- A user listed in `adminUsers` gets admin access to that model.
- A user listed in `writerUsers` can submit ops and lock for that model.
- A user listed in `readerUsers` can read snapshots for that model.
- Catalog-wide admin actions still require the global `admin` role.

9.4 JWT and OIDC setup

9.4.1 Local JWT (dev mode)

For local development with bearer tokens, use `app.identity.mode=oidc` with Quarkus JWT validation:

```
1 app.authz.enabled=true
2 app.identity.mode=oidc
3 mp.jwt.verify.publickey.location=src/test/resources/jwt/publicKey.pem
4 mp.jwt.verify.issuer=https://archimesh.dev
```

Listing 29: Local JWT config

Mint tokens with the dev script:

9. AUTHORIZATION

```
1 ./scripts/mint-dev-jwt.sh --user alice --roles admin,model_writer,model_reader
```

Listing 30: Mint dev JWT

Use the token in requests:

```
1 TOKEN=$(./scripts/mint-dev-jwt.sh --user alice --roles admin)
2 curl -s -H "Authorization: Bearer $TOKEN" \
3   http://localhost:8081/admin/models | jq .
```

Listing 31: Bearer token usage

9.4.2 External OIDC

For production with an external provider (Keycloak, Auth0):

```
1 app.authz.enabled=true
2 app.identity.mode=oidc
3 quarkus.oidc.enabled=true
4 quarkus.oidc.auth-server-url=https://idp.example/realms/archimesh
5 quarkus.oidc.client-id=archimesh-server
6 quarkus.oidc.application-type=service
7 quarkus.oidc.roles.role-claim-path=groups
```

Listing 32: External OIDC config

Concrete provider examples are in `server/examples/`.

9.5 Authorization diagnostics

The endpoint `GET /admin/auth/diagnostics` returns the resolved identity mode, subject, normalized roles and authorization status for the current request. The admin UI exposes this through the **Auth Check** action. This is useful for debugging misconfigured role mappings or proxy headers.

9.6 Configuration summary

Property	Env var	Default
<code>app.authz.enabled</code>	<code>ARCHIMESH_AUTHZ_ENABLED</code>	<code>false</code>
<code>app.identity.mode</code>	<code>ARCHIMESH_IDENTITY_MODE</code>	<code>bootstrap</code>
<code>app.identity.proxy.user</code>	<code>ARCHIMESH_PROXY_USER_HEADER</code>	<code>Forwarded-User</code>
<code>app.identity.proxy.roles</code>	<code>ARCHIMESH_PROXY_ROLES_HEADER</code>	<code>Forwarded-Roles</code>
<code>app.authz.admin-role</code>	<code>ARCHIMESH_AUTHZ_ADMIN_ROLE</code>	<code>admin</code>
<code>app.authz.reader-role</code>	<code>ARCHIMESH_AUTHZ_READER_ROLE</code>	<code>model_reader</code>
<code>app.authz.writer-role</code>	<code>ARCHIMESH_AUTHZ_WRITER_ROLE</code>	<code>model_writer</code>
<code>quarkus.oidc.enabled</code>	<code>ARCHIMESH_QUARKUS_OIDC_ENABLED</code>	<code>false</code>

10 Administration and Supervision

This chapter covers day-to-day operational supervision: the admin UI, audit logging, activity monitoring and session inspection.

10.1 Admin UI

The admin UI is a Svelte application served by Quarkus at `/admin-ui/`. It is organized into seven routes:

Overview Dashboard showing all known models with status summaries, head revisions, element/relationship/view counts and active session counts. Colour-coded status: green (no issues), yellow (stale), red (integrity problems or consistency check failures).

Models Detailed per-model view: catalog metadata, ACL configuration, head revision, estimated counts, tag list and available actions (create tag, rebuild, compact, export, delete). The model selector persists the chosen model in browser storage.

Versions Tag management: list existing tags with description, revision and creation timestamp. Create new tags from current HEAD. Export and import model packages.

Access Identity and authorization: view the current resolved identity, normalized roles, and per-model ACL configuration. Update ACLs for a model. Run authorization diagnostics.

Sessions Live session inspection: active WebSocket sessions per model, with user id, session id, connected-since timestamp and held locks.

Operations Recent model activity: operator-facing events emitted by the server for the selected model (join, disconnect, submit, accept, reject, lock, rebuild, compact, tag, export, import). Recent op batches with batch id, revision range, op count and timestamp.

Audit Admin and WebSocket audit trail: structured log events with resolved user id, action, request context and timestamp. Websocket lifecycle events (open rejected, join, message rejected, close).

The admin UI supports:

- Bootstrap auth inputs for `X-Archimesh-User` and `X-Archimesh-Roles`, plus an optional bearer token for OIDC mode.
- Configurable poll interval (persisted in browser storage; set to 0 to disable auto-refresh).
- When a bearer token is present, admin 401/403 failures explain likely token-expiry or missing-role/model-access problems.

10.2 Audit logging

The server emits two structured audit log streams:

10.2.1 admin_audit

Emitted for admin diagnostic reads, model window reads and mutating admin actions. Each event is an INFO-level log line with a stable JSON payload containing:

- Resolved user id
- Action name

- Request context (model id, endpoint, parameters)
- Timestamp

Rejected admin mutations (409, 400) are audited too.

Operational guidance:

- Ship these JSON lines to a dedicated audit sink or index if relied on for operator review.
- Keep retention shorter than ordinary application logs if dashboard polling is frequent.
- Avoid over-aggressive dashboard refresh intervals in production — repeated window reads produce repeated audit events.
- Treat `admin_audit` as machine-readable output; parse the JSON payload, do not depend on free-form message text.

10.2.2 ws_audit

Emitted for WebSocket lifecycle events:

- Rejected opens
- Successful joins
- Rejected WebSocket messages
- Closes

Configurable via `ARCHIMESH_AUDIT_WEBSOCKET_ACTIONS` (comma-separated action names). Set to an empty value to disable WebSocket audit entirely.

Set `ARCHIMESH_AUDIT_WEBSOCKET_VERBOSE=true` to also emit `WebSocketMessageAccepted` events for accepted `SubmitOps`, `AcquireLock`, `ReleaseLock` and `Presence` messages.

10.3 Supervision checklist

Regular operational supervision tasks:

- Check the admin overview for models with stale head revisions (clients not submitting) or integrity issues.
- Check the sessions page for unexpected disconnects or abandoned sessions.
- Check the operations page for repeated `SubmitOpsRejected*` events — these indicate client problems.
- Check the audit page for unauthorized access attempts.
- Monitor Neo4j disk usage — op-log and materialized state grow with model activity.
- Monitor Kafka topic sizes and consumer lag.
- Periodically run `GET /admin/models/{modelId}/integrity` and check for orphan nodes or missing references.

11 System Management

This chapter covers system management tasks: creating and deleting models, compactions, exports, imports, rebuilds, tags and integrity checks.

11.1 Model lifecycle

11.1.1 Creating a model

Models must be explicitly registered before clients can use them:

```
1 curl -s -X POST http://localhost:8081/admin/models/demo | jq .
```

Listing 33: Register a model

Optionally provide a display name:

```
1 curl -s -X POST http://localhost:8081/admin/models/demo \  
2   -H "Content-Type: application/json" \  
3   -d '{"modelName":"Demo Model"}' | jq .
```

Listing 34: Register with name

The creating user (resolved from the current identity) is seeded as the initial model admin, writer and reader in the ACL.

11.1.2 Deleting a model

```
1 curl -s -X DELETE http://localhost:8081/admin/models/demo | jq .
```

Listing 35: Delete a model

Deletion is refused while the model has active sessions. To override:

```
1 curl -s -X DELETE "http://localhost:8081/admin/models/demo?force=true" | jq .
```

Listing 36: Force delete

11.2 Compaction

Compaction prunes old op-log entries and merge metadata below a configurable retention watermark to bound storage growth.

11.2.1 Running compaction

```
1 curl -s -X POST http://localhost:8081/admin/models/demo/compact | jq .
```

Listing 37: Compact a model

The response reports:

- `modelId`
- `headRevision` — current model head
- `committedHorizon` — revision below which compaction can safely operate
- `retainRevisions` — how many recent revisions are preserved
- `watermarkRevision` — revisions below this are eligible for pruning
- `commitsRemoved` — number of commit nodes deleted
- `opsRemoved` — number of op nodes deleted
- `propertyClocksRemoved` — number of orphan `PropertyClock` nodes removed
- `tombstonesEligible` — number of tombstones eligible under watermark but retained for delete safety
- `clockFieldsEligible` — number of field clock metadata entries eligible under watermark but retained

11.2.2 Compaction safety

Compaction is designed to be safe by default:

- Only commits and ops with revision below the watermark are removed.
- The watermark is derived from the committed revision horizon (`latestCommitRevision - retainRevisions`).
- Tombstones are *never* removed by compaction. They are reported as eligible but retained to prevent stale resurrection.
- Field clocks (`*_lamport`, `*_clientId`) on materialized nodes are retained.
- Only orphan `PropertyClock` nodes (no longer referenced by any materialized entity) below the watermark are pruned.
- Compaction never allows a stale `Create*` or `Update*` to beat a logically newer delete.

11.3 Export

Export produces a self-contained package for one model timeline, suitable for backup or migration.

```
1 curl -s http://localhost:8081/admin/models/demo/export | jq .
```

Listing 38: Export a model

The export package contains:

format Package format identifier (`archi-model-export-v1`).

exportedAt UTC timestamp.

model Model catalog metadata (`modelId`, `modelName`, `headRevision`).

accessControl Per-model ACL snapshot (optional).

snapshot Materialized current state at HEAD. Contains folders, elements, relationships, views, view objects, connections, folder memberships and view object child members.

opBatches Ordered committed op-log history. Each batch contains `modelId`, `opBatchId`, `baseRevision`, `assignedRevisionRange`, timestamp and the ops array with normalized causal metadata.

tags Immutable tags with their historical snapshots.

The package format is documented in detail in `server/EXPORT_FORMAT.md`.

11.3.1 What export preserves

- Model id and name
- Head revision
- Full committed op-log history
- Current materialized state
- Folders and folder memberships
- Views, view objects, connections and notation payloads
- Immutable tags and their historical snapshots
- Model-scoped ACLs

11.4 Import

Import restores a model from an export package.

```
1 curl -s -X POST http://localhost:8081/admin/models/import \  
2   -H "Content-Type: application/json" \  
3   -d @demo-export.json | jq .
```

Listing 39: Import a model

11.4.1 Import behavior

- The `format` field must be `archi-model-export-v1`.
- `model.modelId` must be present.
- Existing models are rejected unless `?overwrite=true`.
- Overwrite is rejected if the target model has active sessions.
- If overwriting, the existing model is deleted first.
- ACLs are restored if present in the package.
- Op batches are restored first, then head revision is set from the imported history.
- Tags are restored after the model timeline.

11.4.2 Overwrite with import

```
1 curl -s -X POST "http://localhost:8081/admin/models/import?overwrite=true" \  
2   -H "Content-Type: application/json" \  
3   -d @demo-export.json | jq .
```

Listing 40: Import with overwrite

11.4.3 Import requirements

For non-empty models, the package must include `opBatches`. Import rejects packages that have a non-zero head revision, non-empty materialized content or historical tags but no op-log history. This prevents restoring current state while silently losing the committed revision history.

11.5 Rebuild

Rebuild clears the materialized state for a model and replays the entire op-log to reconstruct it. This is useful after corruption or for verifying that the op-log faithfully reproduces the materialized state.

```
1 curl -s -X POST http://localhost:8081/models/demo/rebuild | jq .
```

Listing 41: Rebuild a model

Or via the admin endpoint:

```
1 curl -s -X POST http://localhost:8081/admin/models/demo/rebuild-and-status | jq .
```

Listing 42: Rebuild and get status

Rebuild behavior:

1. Clears all materialized nodes for the model (elements, relationships, views, view objects, connections, folders and metadata).
2. Recreates built-in root folders.
3. Replays all committed ops from the op-log in revision order.
4. Updates head revision to match the highest committed revision.
5. Tags are unaffected (they are stored separately).

11.6 Tags

Tags are immutable named pointers to historical revisions. They let you capture known-good states for later reference or checkout.

11.6.1 Creating a tag

```
1 curl -s -X POST http://localhost:8081/admin/models/demo/tags \  
2 -H "Content-Type: application/json" \  
3 -d '{"tagName": "v1.2", "description": "Sprint 12 baseline"}' | jq .
```

Listing 43: Create a tag

The tag captures the current HEAD revision and stores a snapshot of the materialized state at that point.

11.6.2 Reading at a tag

Clients can join at a tagged revision by specifying the tag name as the `ref` in the `Join` message. The snapshot endpoint also accepts `?ref=<tagName>`:

```
1 curl -s "http://localhost:8081/models/demo/snapshot?ref=v1.2" | jq .
```

Listing 44: Read snapshot at tag

Tagged refs are read-only. Writes, locks and presence are only allowed on HEAD.

11.6.3 Tag naming

Use descriptive names that remain meaningful outside the codebase:

- `v1.2` — version numbers
- `milestone-3` — project milestones
- `approved-2026-03-08` — date-stamped approvals

Tag deletion is disabled by default. Re-enable it only with `app.tags.allow-delete=true` if you explicitly want mutable tag administration.

11.7 Integrity checks

The integrity endpoint checks for structural problems in the materialized graph:

```
1 curl -s http://localhost:8081/admin/models/demo/integrity | jq .
```

Listing 45: Check integrity

The response reports:

- Missing references: entities that reference ids not present in the graph (e.g. a view object whose `representsId` points to a deleted element).
- Orphan nodes: materialized nodes not reachable from the model root.
- Dangling relationships: relationships where source or target element is missing.

Integrity issues indicate either a bug in the apply path or manual Neo4j modifications. If issues are found, run a rebuild to reconstruct the materialized state from the op-log.

11.8 Consistency checks

Optional post-commit consistency checks can be enabled with `ARCHIMESH_CONSISTENCY_CHECK=true`. When enabled, the server runs a lightweight consistency verification after each `SubmitOps` commit, comparing the materialized state against the expected state derived from the op-log. Failures are logged at WARN level.

This is disabled by default because it adds latency to every submit. Enable it during debugging or after infrastructure changes.

11.9 ACL management

Per-model ACLs restrict model access to specific users:

```
1 curl -s http://localhost:8081/admin/models/demo/acl | jq .
```

Listing 46: Read ACL

```
1 curl -s -X PUT http://localhost:8081/admin/models/demo/acl \  
2 -H "Content-Type: application/json" \  
3
```

```
3 -d '{"adminUsers":["alice"],"writerUsers":["alice","bob"],"readerUsers":["carol"]}'  
  | jq .
```

Listing 47: Update ACL

When a model has a configured ACL, it overrides generic reader/writer role checks for that model. Catalog-wide admin actions still require the global admin role.

11.10 Status monitoring

Check model health at a glance:

```
1 curl -s http://localhost:8081/admin/models/demo/status | jq .
```

Listing 48: Model status

The response includes:

- `modelId`, `modelName`
- `headRevision`
- Element, relationship, view, folder, view object and connection counts
- Consistency check results
- Compaction status (last compaction timestamp, watermark, commits removed)
- Active session count

12 Troubleshooting

This chapter covers common failure modes, diagnostic approaches and recovery procedures.

12.1 Diagnostic endpoints

Several endpoints help diagnose problems without restarting the server:

Endpoint	Purpose
GET /admin/overview	Check which models exist and their health. Look for stale head revisions or integrity warnings.
GET /admin/models/{modelId}/status	Detailed model status: counts, revision, consistency.
GET /admin/models/{modelId}/integrity	Structural integrity: missing references, orphan nodes.
GET /admin/auth/diagnostics	Authorization troubleshooting: resolved identity, roles, mode.
GET /admin/sessions	Active WebSocket sessions across all models. Check for stale or abandoned sessions.
GET /admin/audit/config	Current audit configuration.

12.2 Common problems

12.2.1 Client cannot connect

Symptom Connection fails with "Failed to connect archimesh websocket".

Possible causes

- Server is not running. Check with

```
curl http://localhost:8081/admin/overview.
```

- Wrong URL. The WebSocket URL should be `ws://<host>:<port>` (not http).
- Authorization failure. Check the auth token validity and role assignments. Use `GET /admin/auth/diagnostics`.
- Model does not exist. Models must be registered via `POST /admin/models/{modelId}` before clients can connect.
- Network firewall blocking WebSocket upgrade.

12.2.2 SubmitOps rejected

Symptom The server broadcasts `Error` messages to the client. The status line shows a conflict banner.

Error codes **REVISION_AHEAD** Client's `baseRevision` exceeds server head. The client is ahead of the server — this usually means the client missed a reconnect after a server state change. Solution: the client should disconnect and reconnect (the server signals this by broadcasting the error to all sessions).

PRECONDITION_FAILED A referenced entity does not exist. The op references an element, relationship, view, view object or connection that is not in the materialized graph and was not created earlier in the same batch. Common causes: deleting an entity on one client while another client edits a diagram object

representing it; submitting a connection create before the relationship is committed. The conflicting outbox entry is dropped and surfaced in the UI.

LOCK_CONFLICT Another client holds a notation lock on one of the targets. The client should retry after releasing and re-acquiring the lock. The conflicting outbox entry is dropped.

12.2.3 Materialized state corrupted

Symptom Integrity check reports missing references or orphan nodes.

Cause Bug in the apply path, manual Neo4j modification or failed partial writes.

Solution Run a rebuild:

```
curl -s -X POST http://localhost:8081/models/demo/rebuild
```

Verification After rebuild, run integrity check again and verify `issues` is empty.

12.2.4 Neo4j connection failure

Symptom Server logs show Neo4j connection errors. All operations fail.

Check Is Neo4j running? `docker compose ps neo4j`. Check the Neo4j browser at `http://localhost:7474`.

Check Are credentials correct? Verify `NEO4J_URI`, `NEO4J_USER` and `NEO4J_PASSWORD` match the running instance.

Check Is the Bolt port accessible? Default is 7687.

12.2.5 Kafka unavailable

Symptom Server logs show Kafka publish warnings. Other clients do not receive real-time updates from the submitter's client.

Impact Submitting clients still receive `OpsAccepted` and can continue editing. The server falls back to direct WebSocket broadcast for `OpsBroadcast`, so clients on the same server instance still receive updates.

Check Is Kafka running? `docker compose ps kafka`. Check the Kafka UI at `http://localhost:8080`.

Check Do the required topics exist? Use

```
docker compose exec kafka rpk topic list --brokers localhost:9092.
```

12.2.6 Outbox filling up

Symptom The Archi status line shows "Outbox near capacity". Local edits are being queued because the client is disconnected or the connection is flapping.

Impact When the outbox reaches 1000 entries, the oldest entries are silently dropped. The warning fires at 800 entries (80%).

Solution Reconnect to the server so the outbox can drain. If connectivity is persistently poor, work in smaller edit batches and reconnect frequently.

Prevention Monitor the server's session page for clients that repeatedly connect and disconnect. Investigate network stability.

12.2.7 Cannot overwrite a model

Symptom Import with `?overwrite=true` returns 409.

Cause Active sessions exist on the target model.

Solution Disconnect all clients from the model, verify with `GET /admin/sessions`, then retry. If a session is stuck, force delete the model and re-import.

12.3 Log analysis

The server emits structured logs. Key log categories:

Logger	Content
<code>org.gautelis.archimesh</code>	Application-level events: submit, accept, reject, join, disconnect, rebuild, compact, export, import, tags.
<code>org.apache.kafka</code>	Kafka client logs (WARN by default).
<code>admin_audit (INFO)</code>	Admin API access audit events.
<code>ws_audit (INFO)</code>	WebSocket lifecycle audit events.

Set `ARCHIMESH_LOG_LEVEL=DEBUG` for detailed flow tracing including op-level detail. Note that `DEBUG` logging of `OpsBroadcast` messages can produce very large log volumes on active models.

Log format:

```
1 yyyy-MM-dd HH:mm:ss.SSS LEVEL [logger.] (thread) message
```

Listing 49: Log format

Audit events include a stable JSON payload with user id, action and request context. Parse the JSON rather than depending on free-form message text.

12.3.1 Client-side log analysis

The client plugin writes to the Archi workspace log (`~/Archi/workspace/.metadata/.log`). Search for:

- `archimesh` — all plugin log entries
- `Connected archimesh` — successful connections
- `SubmitOps received` — server acknowledging submits
- `Queued SubmitOps` — entries moved to outbox
- `Outbox full` — outbox dropping entries
- `Dropping stale` — conflict resolution dropping an entry
- `Error stopping Archimesh` — shutdown errors

The `scripts/check-archimesh-sanity.sh` script scans Archi workspace logs for critical sync issues, integrity warnings and connection errors.

12.4 Recovery procedures

12.4.1 Server restart

If the server is restarted:

1. All WebSocket connections are closed. Clients will show "Disconnected" in the status line.
2. Client outboxes start queuing local edits.
3. When the server is back up, clients must manually reconnect.
4. On reconnect, the client sends a `Join` with its cached revision. The server sends a delta (if the revision gap is within the op-log window) or a full snapshot.
5. The outbox drains automatically after reconnect and revision hints are available.

12.4.2 Neo4j data loss

If the Neo4j database is lost or irrecoverably corrupted:

1. Restore from the most recent export package using `POST /admin/models/import`.
2. If no export exists, the model timeline is lost. Re-register the model and let clients push their local state as new submit batches.
3. Clients with cached models can reconnect after server restore; however, the server will have no op-log history, so the client must discard its cache and cold-start with a snapshot from the server (which will be empty for a newly registered model).

12.4.3 Full system recovery

In a disaster scenario where both Neo4j and the server are lost:

1. Restore infrastructure: `docker compose up -d`.
2. Apply Neo4j schema: `docker compose exec -T neo4j cypher-shell ... < neo4j/schema.`
3. Start the server.
4. Import the most recent model exports.
5. Clients reconnect and receive snapshots from the restored server.

12.5 Performance tuning

- **Neo4j heap:** The materialized graph and op-log grow with model activity. Monitor Neo4j heap usage and increase `NEO4J_dbms_memory_heap_max__size` as needed.
- **Kafka retention:** Long Kafka retention on ops topics consumes disk. If rebuild is done from Neo4j (not Kafka), set shorter retention for ops topics.
- **Compaction frequency:** Run compaction periodically on active models to bound op-log growth. The compaction reports the number of pruned commits and remaining eligible metadata.
- **Poll interval:** The admin UI polls the server for status updates. Set the poll interval to 0 to disable auto-refresh if dashboard traffic is a concern.

- **Consistency checks:** Keep `ARCHIMESH_CONSISTENCY_CHECK` disabled in production unless debugging. It adds latency to every submit.

13 Script Reference

This appendix documents all scripts in the `scripts/` directory.

13.1 Build and install

build-archimesh-client.sh Convenience wrapper that calls `build-client-standalone.sh`.

build-client-standalone.sh Builds the client plugin using Tycho/Maven with the target platform.

build-admin-web.sh Builds the Svelte admin UI and copies output to the server's static resources directory.

install-archimesh-plugin.sh Installs the built client plugin JAR into the local Archi installation's dropins directory.

sync-archimesh-client.sh Syncs client plugin source changes without a full rebuild.

prepare-client-target-platform.sh Prepares the Eclipse target platform bundles required for client plugin compilation.

mint-dev-jwt.sh Mints a local dev JWT signed with the test private key.

Usage: `./mint-dev-jwt.sh -user alice -roles admin,model_writer`

Options: `-expires-in <seconds>`, `-audience <aud>`, `-print-payload`.

Env overrides: `DEV_JWT_PRIVATE_KEY`, `DEV_JWT_ISSUER`, `DEV_JWT_EXPIRES_IN`, `DEV_JWT_AUDIENCE`.

13.2 Development infrastructure

dev-up.sh Starts Docker Compose infrastructure (Neo4j + Kafka) and optionally the server. Usage: `./dev-up.sh [background]`. Supports OIDC mode via `ARCHIMESH_IDENTITY_MODE=oidc`.

dev-down.sh Stops Docker Compose infrastructure.

13.3 Validation and gate checks

validate-local.sh Central validation entry point.

Usage: `./validate-local.sh <mode>`

Modes:

fast Lightweight hardening and parity checks (no Docker infra needed). Runs `check-crdt-hardening.sh` and `check-notation-parity.sh`.

gate Full client + server CRDT gate via `crdt-gate.sh`.

infra Full gate with local Kafka/Neo4j-backed integration coverage. Sets `RUN_LOCAL_INFRA_IT=true`.

ws Full gate with websocket end-to-end coverage. Sets `RUN_WS_E2E_IT=true`.

crdt-gate.sh Runs the CRDT test suite end-to-end. First runs hardening contract checks, then installs the client plugin artifact, runs client CRDT tests (`OpMapperNotationInventoryTest`, `OpMapperConnectionE`, `CrdtEntityMergeTest`, `CrdtPropertyMergeTest`, `CrdtOrSetTest`), then runs server CRDT tests (`ArchimeshServiceTest`, optionally `LocalInfraIntegrationTest` when `RUN_LOCAL_INFRA_IT=true`).

check-crdt-hardening.sh Preflight contract check that verifies presence of core CRDT test files, gate scripts and spec references. Fails fast when test infrastructure drifts.

check-notation-parity.sh Compares notation key inventory across: schema definitions (`schemas/ops.json`), server runtime whitelist constants (`ArchimeshService`) and client inventory test expectations (`OpMapperNotation`). Ensures all three agree on supported notation fields.

check-archimesh-sanity.sh Scans Archi workspace logs for critical sync issues, integrity warnings and connection errors. Useful for diagnosing client-side problems.

13.4 Log monitoring

tail-archimesh-logs.sh Tails Archi workspace logs with archimesh-related filtering. Useful for real-time debugging during client plugin development.

14 Wire Protocol and Schema Reference

This appendix documents the JSON schemas that define the Archimesh wire protocol contract. The authoritative schema files live in `schemas/`.

14.1 Schema files

File	Purpose
<code>common.json</code>	Shared type definitions: <code>ModelId</code> , <code>UserId</code> , <code>SessionId</code> , <code>EntityId</code> , <code>RFC3339</code> , <code>Causal</code> , <code>Actor</code> , <code>RevisionRange</code> , <code>JsonObject</code> .
<code>ops.json</code>	Operation type definitions for all supported op types: <code>create</code> , <code>update</code> , <code>delete</code> for elements, relationships, views, view objects, connections, folders; property and membership operations; plus strict notation JSON objects for view objects and connections.
<code>op-batch.json</code>	Op batch envelope: <code>SubmitOps</code> with <code>baseRevision</code> , <code>opBatchId</code> , <code>actor</code> and <code>ops</code> array.
<code>lock-event.json</code>	Lock event payload: <code>action</code> (<code>acquired</code> , <code>released</code> , <code>expired</code>), <code>actor</code> , <code>targets</code> , <code>timestamp</code> .
<code>presence.json</code>	Presence payload: <code>modelId</code> , <code>userId</code> , <code>sessionId</code> , <code>viewId</code> , <code>selection</code> array, <code>cursor</code> object.
<code>notation-contract.md</code>	Human-readable contract describing the notation field boundaries and parity requirements.

14.2 Message flow summary

Direction	Type	Description
Client → Server	<code>Join</code>	Request checkout. Optional <code>lastSeenRevision</code> and <code>ref</code> .
Client → Server	<code>SubmitOps</code>	Submit one op batch. <code>baseRevision</code> , <code>opBatchId</code> , <code>ops</code> .
Client → Server	<code>AcquireLock</code>	Acquire notation lock on targets. <code>ttlMs</code> .
Client → Server	<code>ReleaseLock</code>	Release notation lock on targets.
Client → Server	<code>Presence</code>	Broadcast cursor position and selection on a view.
Server → Client	<code>CheckoutSnapshot</code>	Full materialized model state.
Server → Client	<code>CheckoutDelta</code>	Incremental ops since <code>lastSeenRevision</code> .
Server → Client	<code>OpsAccepted</code>	Batch commit confirmation with assigned revision range.
Server → Client	<code>OpsBroadcast</code>	Remote batch broadcast (Kafka fan-out or direct fallback).
Server → Client	<code>LockEvent</code>	Lock state change notification.
Server → Client	<code>PresenceBroadcast</code>	Remote cursor/selection update.
Server → Client	<code>Error</code>	Rejection or failure with <code>code</code> and <code>message</code> .

14.3 Common type definitions

14.3.1 Causal metadata

```
1 {
2   "clientId": "<stable client or session identifier>",
3   "lamport": <monotonic integer per client>,
4   "opId": "<globally unique operation identity>"
5 }
```

Listing 50: Causal object

14.3.2 Actor

```
1 {
2   "userId": "<user identifier>",
3   "sessionId": "<session identifier>"
4 }
```

Listing 51: Actor object

14.3.3 Revision range

```
1 {
2   "from": <inclusive start revision>,
3   "to": <inclusive end revision>
4 }
```

Listing 52: Revision range object

14.4 Op batch envelope

```
1 {
2   "type": "SubmitOps",
3   "payload": {
4     "baseRevision": <client revision>,
5     "opBatchId": "<UUID>",
6     "actor": { "userId": "...", "sessionId": "..." },
7     "ops": [
8       { "type": "CreateElement", ... },
9       { "type": "UpdateViewObjectOpaque", ... }
10    ]
11  }
12 }
```

Listing 53: SubmitOps payload

14.5 Notation JSON objects

View object and connection notation are validated against strict schemas (`additionalProperties: false`):

14.5.1 ViewObjectNotationJson

Supported fields: x, y, width, height, type, alpha, lineAlpha, lineWidth, lineStyle, textAlignment, textPosition, gradient, iconVisibleState, deriveElementLineColor, fillColor, lineColor, font, fontColor, iconColor, imagePath, imagePosition, name, documentation.

14.5.2 ConnectionNotationJson

Supported fields: type, nameVisible, textAlignment, textPosition, lineWidth, name, lineColor, font, fontColor, documentation, bendpoints.

14.6 Error codes

Code	Meaning
REVISION_AHEAD	Client baseRevision exceeds server head. Client must rejoin.
PRECONDITION_FAILED	Referenced entity does not exist, or notation contains unknown keys.
LOCK_CONFLICT	Another client holds a conflicting notation lock.
MODEL_NOT_FOUND	The requested model is not registered.
REF_NOT_FOUND	The requested tag name does not exist.
REF_READ_ONLY	Write attempted on a tagged (read-only) ref.

14.7 Export/import format

The admin export format is a JSON package containing:

```
1 {
2   "format": "archi-model-export-v1",
3   "exportedAt": "<UTC timestamp>",
4   "model": { "modelId": "...", "modelName": "...", "headRevision": 42 },
5   "accessControl": { "configured": true, "adminUsers": [...], ... },
6   "snapshot": { "format": "archimate-materialized-v1", ... },
7   "opBatches": [ { "modelId": "...", "opBatchId": "...", ... } ],
8   "tags": [ { "tagName": "...", "revision": 24, "snapshot": {...} } ]
9 }
```

Listing 54: Export package structure

The package format is documented in full in `server/EXPORT_FORMAT.md`.